

Department of Computer Science

Course: AI Programming

Lab Number: Lab 8: Dictionaries and Functions in Python

Lab Objectives

By the end of this lab, students should be able to:

1. Create and update Python dictionaries.
2. Access dictionary values using keys.
3. Write functions that receive dictionaries.
4. Write functions that return dictionaries.
5. Use `**kwargs` to create dictionaries.

Task 1: Create a Student Dictionary

Create a dictionary called `student`:

```
student = {  
    "name": "Ahmad",  
    "id": "2024001",  
    "major": "Artificial Intelligence",  
    "grade": 85  
}
```

Print each value in this format:

Student Name: Ahmad

Student ID: 2024001

Major: Artificial Intelligence

Grade: 85

Task 2: Add Student Status

Add a new key called "status".

If the grade is greater than or equal to 60, the status should be "Pass". Otherwise, it should be "Fail".

Expected output:

```
{'name': 'Ahmad', 'id': '2024001', 'major': 'Artificial Intelligence', 'grade': 85, 'status': 'Pass'}
```

Task 3: Update Student Grade

Ask the user to enter a new grade.

Update the value of "grade" in the dictionary.

Then update "status" again based on the new grade.

Example:

Enter new grade: 55

```
{'name': 'Ahmad', 'id': '2024001', 'major': 'Artificial Intelligence', 'grade': 55, 'status': 'Fail'}
```

Task 4: Function to Calculate Status

Write a function called `get_status(grade)`.

The function should return:

- "Pass" if grade is 60 or more
- "Fail" otherwise

Example:

```
print(get_status(85))
```

```
print(get_status(45))
```

Expected output:

Pass

Fail

Task 5: Function to Print Student Info

Write a function called `print_student_info(student)`.

The function should receive a dictionary and print:

Name: Ahmad

ID: 2024001

Major: Artificial Intelligence

Grade: 85

Status: Pass

Task 6: Function to Update Grade

Write a function called `update_grade(student, new_grade)`.

The function should:

Update the student grade.

Update the student status using `get_status()`.

Return the updated dictionary.

Example:

```
student = update_grade(student, 92)
```

```
print(student)
```

Task 7: Create Student Using Function

Write a function called `create_student(name, student_id, major, grade)`.

The function should return a dictionary containing:

```
{  
    "name": name,  
    "id": student_id,  
    "major": major,
```

```
    "grade": grade,
    "status": get_status(grade)
}
```

Create two students using this function.

Task 8: Use ****kwargs**

Write a function:

```
def create_student_kwargs(**kwargs):
    return kwargs
```

Use it to create a student:

```
student2 = create_student_kwargs(
    name="Sara",
    id="2024002",
    major="Computer Science",
    grade=92
)
```

Then add the "status" key using `get_status()`.

Task 9: Dictionary of Students

Create a dictionary called `students`. Each key should be a student ID, and each value should be another dictionary. Example:

```
students = {
    "2024001": {
        "name": "Ahmad",
        "major": "AI",
        "grade": 85,
        "status": "Pass"
    },

```

```
"2024002": {  
    "name": "Sara",  
    "major": "CS",  
    "grade": 92,  
    "status": "Pass"  
}  
}
```

Print all students using a loop.

Task 10: Search Student by ID

Write a function called *find_student(students, student_id)*.

The function should:

Return the student dictionary if the ID exists.

Return "Student not found" if the ID does not exist.

Example:

```
print(find_student(students, "2024001"))  
print(find_student(students, "2024999"))
```

Submission Requirements

Submit one Python file named:

lab_8_studentID.py